

Programmation Orientée Objet pour le Calcul Scientifique : Part II Deep Water

Organisation du cours

Sous forme d'**Item**, se raccrochant à trois catégories :

- Outils et Langage (Tool)
- **Gestion de la mémoire (Mem)**
- Orienté objet (OO)
- Programmation statique (Static)

Vous n'êtes pas seul : <https://en.cppreference.com/w/> est votre allié...

Item 16 : copier sans copier

Le but de cet item est d'introduire (un peu grossièrement) les notions :

- de *rvalue*
- de *rvalue reference* `&&` dans les signatures de fonctions/méthodes
- de *std::move*

Toutes ces notions vont nous aider à **déplacer la mémoire** d'un objet/variable

- afin d'éviter des copies inutiles et coûteuses
- ces notions ont donc leur intérêt dans une optique *Calcul Scientifique*

Item 16 : copier sans copier

Une *rvalue* 1/5

- est une valeur que l'on ne peut pas nommer (temporaire, retour de fonction...)
- ne peut s'utiliser que dans le membre de droite d'une affectation (right-value)
- n'a pas d'adresse utilisable

```
int a = 4, b = 2; // 4 & 2 are rvalues
int c = a+b; // a+b = rvalue
std::string s = "hello"; // "hello" = rvalue
int d = a++; // a++ = rvalue

int_ptr = &(a+b); // ERROR : cannot take rvalue address

int func() {return 42;}

auto a = func(); // func() = rvalue
```

Item 16 : copier sans copier

Une *rvalue* 2/5

s'oppose à la classique **lvalue** (left value), qui elle peut :

- être nommée
- apparaître à droite ou à gauche d'une affectation
- avoir une adresse

```
int a = 4, b = 2; // a & b are lvalues
int c = a+b; // c is a lvalue
std::string s = "hello"; // s is a lvalue

int* int_ptr = &a; // OK can take lvalue address
```

Item 16 : copier sans copier

Une *rvalue* 3/5

Petit Quiz : rvalue ou lvalue ??

```
struct A{};
A a{}; // a ?
A{}; // A{} ?
int global = 1;
int& func() {return global;}
func(); // func()
```

Comment savoir ? Impossible d'affecter dans une rvalue :

```
int i = 1 , j = 2;
i + 2 = j; // Error
3 = j; // Error
```

Item 16 : copier sans copier

Une *rvalue* 4/5

peut être utilisée pour initialiser :

- une *lvalue*
- une référence **constante** de *lvalue*

```
int a = 4, b = 2; // lvalues initialized by rvalues
int c = a + b; // lvalue initialized by rvalue
std::string s = "hello"; // lvalue initialized by rvalue
int const& d = a+b; // lvalue const reference initialized by rvalue
```

- une référence de *lvalue* ne peut-être initialisée que par une *lvalue*

```
int& d = a; // lvalue reference initialized by lvalue
int& e = 3; // ERROR
```

Item 16 : copier sans copier

Une *rvalue* 5/5

Encore un quiz...

- une *rvalue* ne peut pas être modifiée ?
- on peut toujours faire une affectation dans une *lvalue* ?

```
// Hint for rvalue  
A{}.method();
```

```
// Hint for lvalue  
int const c = 3;
```

Item 16 : copier sans copier

La référence de *rvalue* `&&`

Commençons par un exemple : y-a-t-il une recopie du vecteur ?

```
std::vector<int> a = std::vector<int>{1,2,3,4}; // lvalue from rvalue
```

La réponse est **non** ! ouf...Mais pourquoi ? Quel est l'opérateur appelé ?

`std::vector<T>::operator=(std::vector<T> const& rhs)` ? Non car :

- il attend une référence de *lvalue* et non de *rvalue*
- cet opérateur fait d'ailleurs une copie...

La définition de la *rvalue-reference* `&&` permet de normaliser ce comportement

```
std::vector<T>::operator=(std::vector<T>&& rhs)
```

- l'opérateur prend une *rvalue* et "vole" sa mémoire

Item 16 : copier sans copier

La fonction `std::move` 1/2

Peut-on "voler" la mémoire d'une *lvalue* ?

```
struct A {  
    std::vector<int> data;  
};  
int main() {  
    std::vector<int> vec {1,2,3,4};  
    // don't want to use vec anymore but want to store it in A.data  
    A a{};  
    a.data = vec; // copy...  
}
```

Oui grâce à la méthode `std::move` qui "transforme" une *lvalue* en *rvalue*

Item 16 : copier sans copier

La fonction `std::move` 2/2

Peut-on "voler" la mémoire d'une *lvalue* ?

- Oui grâce à la méthode `std::move` qui "transforme" une *lvalue* en *rvalue*

```
struct A {  
    std::vector<int> data;  
};  
int main() {  
    std::vector<int> vec {1,2,3,4};  
    // don't want to use vec anymore but want to store it in A.data  
    A a{};  
    a.data = std::move(vec); // no copy...  
}
```

Attention, après un `std::move(var)` `var` n'est plus utilisable

Item 16 : copier sans copier

La fonction `std::move` 2/2

Peut-on "voler" la mémoire d'une *lvalue* ?

- Oui grâce à la méthode `std::move` qui "transforme" une *lvalue* en *rvalue*

Exo 16.1 construisez un tableau et "voler" sa mémoire dans un autre tableau. Vérifier que ce premier tableau n'a plus de mémoire.

Exo 16.2 créez une fonction qui prend un vecteur par référence de *rvalue* et qui l'affiche à la console. Appelez-la dans un `main`, à partir d'une *rvalue* et voyez ce qui se passe lorsque vous l'appellez à partir d'une *lvalue*.

Exo 16.3 reprenez l'exo 16.2 et construisez dans votre fonction un vecteur à partir de votre référence de *rvalue*. Attention il ne devra pas y avoir de copie.

Item 16 : copier sans copier

Les fonctions implicites des classes sont enrichies avec des `&&`

Rappelez-vous l'*item 10*

```
struct A{ // ce que vous ne voyez pas
    A() {}; // constructor
    ~A() {}; // destructor
    A(A const& a) {m_i = a.i;} // copy constructor
    A& operator= (A const& a) {m_i = a.i;} // copy assignment
    int m_i = 0;
}; // move operators kept under silence for the moment
```

Sont maintenant automatiquement ajoutées les opérations de *move*

```
A(A&& a) : m_i{std::move(a.i)} {} // move constructor
A& operator= (A&& a) {m_i = std::move(a.i);} // move assignment
```

Item 16 : copier sans copier

Un petit exercice : **Exo 16.4** :

- écrivez une classe contenant explicitement les sept méthodes implicites.
- utilisez toutes ces méthodes dans une fonction `main`.
- vous pouvez repartir de l'**Exo 10.1** et ajouter les opérations de move
- pour voir quelles sont les méthodes appelées, ajoutez un affichage de la signature

Item 17 : les pointeurs intelligents

La gestion de zones mémoires via des pointeurs n'est pas sans risque :

- oubli du `delete` = fuite mémoire ; double `delete` = crash
- manipulation d'un pointeur nul
- erreur dans l'arithmétique de pointeur

Le C++, à partir de la norme 11 propose donc des classes de *smart pointer*

- `std::unique_ptr<T>`
- `std::shared_ptr<T>`
- `std::weak_ptr<T>`

Les *smart pointers* désallouent automatiquement la mémoire pour vous

Item 17 : les pointeurs intelligents

L'unique `std::unique_ptr<T>` 1/2

- la donnée pointée est unique et ne peut-être copiée
- la donnée pointée est détruite quand le pointeur est détruit
- la donnée peut être transférée à un autre `unique_ptr` via un `std::move`

```
#include <memory>

int main()
{
    {
        std::unique_ptr<int> int_ptr(new int{3});
        std::unique_ptr<int> int_ptr2 = int_ptr; // ERROR
        std::unique_ptr<int> int_ptr3 = std::move(int_ptr); // OK. int_ptr undefined
    } // pointed data *int is deleted
}
```

Item 17 : les pointeurs intelligents

L'unique `std::unique_ptr<T>` 2/2

- comme dans un pointeur classique, la donnée est accessible via `operator*`
- le pointeur stocké dans `std::unique_ptr<T>` est accessible par la méthode `get()`
- un `std::unique_ptr<T>` peut être créé via la méthode `std::make_unique(ptr)`

```
std::unique_ptr<int> int_ptr = std::make_unique<int>(3);  
std::cout << *int_ptr << std::endl;  
int* ptr = int_ptr.get();  
std::cout << *ptr << std::endl;
```

Exo 17.1 : Créer un `unique_ptr<std::string>` pour manipuler une chaîne de caractère. Affichez le contenu de la chaîne et l'adresse du pointeur `std::string*`. Transférez votre pointeur à un nouveau `unique_ptr` et refaites les mêmes affichages.

Item 17 : les pointeurs intelligents

Le partage avec `std::shared_ptr<T>` 1/3

- la donnée pointée peut-être partagée entre plusieurs `std::shared_ptr<T>`
- la donnée pointée est détruite quand le 'dernier' `shared_ptr` est détruit

```
#include <memory>
int main()
{
    {
        std::shared_ptr<int> int_ptr(new int{3}); // int allocated
        {
            std::shared_ptr<int> int_ptr2 = int_ptr; // OK, no new allocation
        } // int_ptr2 released but the data is still allocated
    } // pointed data *int is deleted
}
```

Item 17 : les pointeurs intelligents

Le partage avec `std::shared_ptr<T>` 2/3

- L'accès à la donnée pointée ou au pointeur est identique au `unique_ptr`
 - `operator*` pour la donnée et méthode `get()` pour le pointeur
- un `shared_ptr` peut être créé par la fonction `std::make_shared(ptr)`

```
std::shared_ptr<int> int_ptr = std::make_shared<int>(3);  
std::cout << *int_ptr << std::endl;  
int* ptr = int_ptr.get();  
std::cout << *ptr << std::endl;
```

Item 17 : les pointeurs intelligents

Le partage avec `std::shared_ptr<T>` 3/3

- le `shared_ptr` utilise un mécanisme de compteur de référence
- la donnée est allouée une seule fois et les `shared_ptr` se partagent une référence
- si la donnée est modifiée via l'un des `shared_ptr` elle est modifiée pour tous

```
std::shared_ptr<int> int_ptr = std::make_shared<int>(3);  
std::shared_ptr<int> int_ptr2 = int_ptr;  
(* int_ptr)++; // increment value  
std::cout << *int_ptr2 << std::endl; // 4
```

Exo 17.2 Créer un `shared_ptr<double>` et copiez-le dans un autre `shared_ptr`. Modifiez la valeur du `double` depuis l'un des `shared_ptr` et vérifiez qu'elle est bien modifiée dans le second.

Item 17 : les pointeurs intelligents

l'accès si possible avec `std::weak_ptr<T>` 1/2

- le `weak_ptr` permet d'accéder à une donnée contenue dans un `shared_ptr`
- sans pour autant contribuer au comptage de référence
- la donnée pointée peut donc être désallouée

```
std::weak_ptr<int> int_weak_ptr;  
{  
    std::shared_ptr<int> int_ptr = std::make_shared<int>(3);  
    int_weak_ptr = int_ptr;  
    if ( !int_weak_ptr.expired()) {std::cout << "weak_ptr data alive."<< "\n";}  
}  
if (int_weak_ptr.expired()) {std::cout << "weak_ptr data deleted."<< "\n";}
```

Item 17 : les pointeurs intelligents

l'accès si possible avec `std::weak_ptr<T>` 2/2

- le `weak_ptr` ne permet pas un accès direct à la donnée pointée
- il faut d'abord recopier dans un `shared_ptr` avec la méthode `lock`

```
std::shared_ptr<int> int_sptr = std::make_shared<int>(3);
std::weak_ptr<int> int_wptr = int_sptr;
// Try to access pointed value from weak_ptr
if ( !int_wptr.expired() ) {
    auto shared_ptr_tmp = int_wptr.lock();
    std::cout << "weak_ptr data " << *shared_ptr_tmp << "\n";
}
```

Item 17 : les pointeurs intelligents

Utilisation d'un destructeur spécifique 1/2

- par défaut un `delete` sur l'objet pointé
- possibilité de fournir un destructeur spécifique
- sous la forme d'une fonction générique (Foncteur, λ ...)

```
// unique_ptr case
auto my_deleter = [](PointedObject* ptr)
{
    std::cout << "Delete PointedObject";
    delete ptr;
}

using unique_ptr_with_del = std::unique_ptr<PointedObject, decltype(my_deleter)>;
unique_ptr_with_del u_ptr(new PointedObject{}, my_deleter);
}
```

Item 17 : les pointeurs intelligents

Utilisation d'un destructeur spécifique 2/2

- par défaut un `delete` sur l'objet pointé
- possibilité de fournir un destructeur spécifique
- sous la forme d'une fonction générique (Foncteur, λ ...)

```
// shared_ptr case
auto my_deleter = [](PointedObject* ptr)
{
    std::cout << "Delete PointedObject";
    delete ptr;
};

std::shared_ptr<PointedObject> s_ptr(new PointedObject{}, my_deleter);
```

Item 17 : les pointeurs intelligents

- **Exo 17.3 1/4:** Maille et face d'un maillage. On souhaite créer un use case d'une maille contenant un pointeur vers sa face. Les opérations seront les suivantes :
 - création d'une maille (classe `Cell`)
 - affichage de l'id de sa face (champ `int` dans la classe `Face`)
 - destruction de la maille

On souhaite réaliser trois scénarios (dans trois fonctions que vous appellerez depuis votre `main()`).

Item 17 : les pointeurs intelligents

- **Exo 17.3 2/4:** Maille et face d'un maillage. On souhaite créer un use case d'une maille contenant un pointeur vers sa face.
 - **Scénario 1 :** la maille est la seule à posséder le pointeur de sa face. Elle ne le partage pas. Dans ce scénario, vous écrirez le cas où la maille transfère la propriété de sa face à une nouvelle maille avant d'être détruite.

Item 17 : les pointeurs intelligents

- **Exo 17.3 3/4:** Maille et face d'un maillage. On souhaite créer un use case d'une maille contenant un pointeur vers sa face.
 - **Scénario 2 :** la maille peut partager sa face et celle-ci peut rester allouée même après la suppression de la cellule. Dans l'exercice vous recopierez la face de la maille dans un pointeur extérieur à la maille qui devra 'survivre' à la destruction de la maille. Pour le vérifier on affichera l'id de la face après la destruction de la maille.

Item 17 : les pointeurs intelligents

- **Exo 17.3 4/4** : Maille et face d'un maillage. On souhaite créer un use case d'une maille contenant un pointeur vers sa face.
 - **Scénario 3** : la maille peut toujours partager sa face que l'on va à nouveau stocker dans un pointeur. En revanche on veut que cette face soit effectivement détruite lorsque la maille est détruite. On vérifiera donc que ce pointeur a expiré après la destruction de la maille.